13. Python: Dictionaries

"Every Key has its Value."

Previous Section:

</> 12. Python: Sets

Contents:

Another extremely important data structure in Python is the dictionary. Unlike lists, tuples, or sets, dictionaries store data using a **key-value relationship**. Instead of accessing data using indexes, we retrieve values using their keys.

Dictionaries are very useful when we want to organize related information, perform fast lookups, or represent real-world objects such as students, users, products, or months.

1. What is a Dictionary?

A dictionary is a built-in Python data structure that stores data as **key-value pairs**.

Example:

```
student = {"name": "Iris",
           "age": 20, "major":
           "Computer Science"}

print(student)
```

{'name': 'Iris', 'age': 20, 'major': 'Computer Science'}

Properties of Dictionaries

- **Keys must be immutable:** Dictionary keys cannot change. Therefore, keys can be: integers, strings, floats, or immutable tuples.

```
data = {1: "Integer Key",
        "name": "String Key",
        3.14: "Float Key",
         (1, 2): "Tuple Key"}

print(data)
```

{1: 'Integer Key', 'name': 'String Key', 3.14: 'Float Key', (1, 2): 'Tuple Key'}

However, lists and sets cannot be used as keys because they are mutable.

```
# Invalid dictionary keys
data = {[1, 2]: "List Key", {1, 2}: "Set Key"}
print(data)
```

TypeError: unhashable type: 'list'

- **Duplicate keys are not allowed:** If duplicate keys exist, Python only keeps the last value.

```
dictionaries = {"a": 1, "a": "b"}
print(dictionaries)
```

{'a': 'b'}

- **Values can be any data type:** Dictionary values are very flexible.

```
data = {"name": "Iris",
        "age": 20,
        "scores": [90, 85, 100],
        "status": True}

print(data)
```

Creating dictionaries using `{}` and `dict()`

- We can create dictionaries using curly brackets:

```
student = {"name": "Iris", "age": 20}
print(student)
```

{'name': 'Iris', 'age': 20}

- Or using `dict()`:

```
student = dict(name="Iris", age=20)
print(student)
```

{'name': 'Iris', 'age': 20}

Creating a dictionary from two lists

- We can combine two lists using `zip()` and `dict()`.

```
keys = ["Jan", "Feb", "Mar"]
values = [1, 2, 3]

month_dict = dict(zip(keys, values))
print(month_dict)
```

{'Jan': 1, 'Feb': 2, 'Mar': 3}

Creating a dictionary using a set as keys

- Sets can also be used to create dictionaries.

```
keys = {"A", "B", "C"}
new_dict = dict.fromkeys(keys, 0)

print(new_dict)
```

{'A': 0, 'B': 0, 'C': 0}

Using `enumerate()`

- `enumerate()` automatically generates indexes.

```
names = ["Iris", "Luna", "Aether"]
data = dict(enumerate(names))

print(data)
```

{0: 'Iris', 1: 'Luna', 2: 'Aether'}

2. Methods in Dictionaries

- `dict.keys()`: Returns all keys in the dictionary.

```
month_dict = {'Jan': 1, 'Feb': 2, 'Mar': 3}
print(month_dict.keys())
```

dict_keys(['Jan', 'Feb', 'Mar'])

- `dict.values()` : Returns all values in the dictionary.

```
month_dict = {'Jan': 1, 'Feb': 2, 'Mar': 3}
print(month_dict.values())
```

dict_values([1, 2, 3])

- `dict.items()` : Returns all key-value pairs as tuples.

```
month_dict = {'Jan': 1, 'Feb': 2, 'Mar': 3}
print(month_dict.items())
```

dict_items([('Jan', 1), ('Feb', 2), ('Mar', 3)])

- **Iterating through a dictionary:** We commonly use `items()` with loops.

```
month_dict = {'Jan': 1, 'Feb': 2, 'Mar': 3}

for key, value in month_dict.items():
    print(key, value)
```

```
Jan 1
Feb 2
Mar 3
```

- `dict.get()`: Returns the value for a specific key.

```
month_dict = {'Jan': 1, 'Feb': 2, 'Mar': 3}
print(month_dict.get('Mar'))
```

```
3
```

We can also provide a default value if the key does not exist.

```
month_dict = {'Jan': 1, 'Feb': 2, 'Mar': 3}
print(month_dict.get('XYZ', 'Not Found'))
```

```
Not Found
```

- `dict.fromkeys()` : Creates a new dictionary using sequential keys.

```
new_dict = dict.fromkeys(['A', 'B', 'C'], 0)
print(new_dict)
```

```
{'A': 0, 'B': 0, 'C': 0}
```

- `dict.pop()` : Removes a specific key and returns its value.

```
month_dict = {'Jan': 1, 'Feb': 2, 'Mar': 3}
removed_value = month_dict.pop('Jan')

print(removed_value)
print(month_dict)
```

```
1
{'Feb': 2, 'Mar': 3}
```

- `dict.popitem()` : Removes and returns the last inserted key-value pair.

```
month_dict = {'Jan': 1, 'Feb': 2, 'Mar': 3}
last_item = month_dict.popitem()

print(last_item)
print(month_dict)
```

```
('Mar', 3)
{'Jan': 1, 'Feb': 2}
```

- `dict.setdefault()` : Returns the value of a key. If the key does not exist, Python inserts the key with a default value.

```
month_dict = {'Feb': 2, 'Mar': 3}
value = month_dict.setdefault('Jan', 1)

print(value)
print(month_dict)
```

```
1
{'Feb': 2, 'Mar': 3, 'Jan': 1}
```

- `dict.update()` : Updates the dictionary using another dictionary.

```
month_dict = {'Jan': 1, 'Feb': 2}
month_dict.update({'Mar': 3, 'Apr': 4})

print(month_dict)
```

{'Jan': 1, 'Feb': 2, 'Mar': 3, 'Apr': 4}

- `len(dict.keys())` : Returns the number of keys.

```
month_dict = {'Jan': 1, 'Feb': 2, 'Mar': 3}
print(len(month_dict.keys()))
```

3

- `len(dict.values())` : Returns the number of values.

```
month_dict = {'Jan': 1, 'Feb': 2, 'Mar': 3}
print(len(month_dict.values()))
```

3

- `sum(dict.keys())` : Sums all keys.

```
num_dict = {1: 10, 2: 20, 3: 30}
print(sum(num_dict.keys()))
```

6

- `sum(dict.values())` : Sums all values.

```python
num_dict = {1: 10, 2: 20, 3: 30}
print(sum(num_dict.values()))
```

```
60
```

Summary

Dictionaries are one of the most powerful and commonly used data structures in Python.

Unlike lists or tuples, dictionaries store data using key-value pairs, making them extremely efficient for searching and organizing information.

Key points:

- Keys must be unique and immutable.
- Values can be any data type.
- Dictionaries are very fast for lookups.
- Many useful methods exist for updating, retrieving, and managing data.

Overall, dictionaries are widely used in: databases, APIs, web applications, machine learning, and almost every large Python project.

References

https://www.geeksforgeeks.org/python/python-dictionary/

https://www.programiz.com/python-programming/dictionary

https://realpython.com/python-dicts/

Next Section:

</> 14. Python: Strings